Zenith: learning executable investigations

Will Cook • 13 October 2026

Abstract. What should accumulate when a language-model investigator learns from failure? Zenith retains an executable procedure, its claimed scope, measured counterexamples and versioned dependencies. An investigation returns supported findings alongside unresolved evidence obligations. A measured failure can prompt a replacement computation; complete replay decides whether to retain it. We synthesise five independently returned prototypes into an implemented test of model-authored programs, using operational time-series measurements rather than unsupported market prediction. The experiment separates one-shot compilation, repair, frozen reuse and source-schema adaptation, with costs and failed attempts retained. The contribution is a testable implementation and evidence, not a claim of new general intelligence or an unprecedented learning architecture.

1. Knowledge that can do work. A useful retained object should answer a later question, acquire missing evidence, or determine that a conclusion remains unsupported. More text or more graph nodes need not achieve any of these. We therefore make the editable object an investigation program P, with context c and an acquisition interface Q:

$$P(c, Q) \longmapsto (F, U, \rho).$$

F contains established findings, U contains unresolved checks, and ρ is a numerical comparison when identifiable from the supplied evidence. The executed graph records acquired tables and their hashes; it is a computation trace, not a graph of causes in the external world.

A concrete operational task. Child-operation intervals describe requests before and after a deployment. The quantity contract distinguishes active time, elapsed time and billed work:

$$q_{\text{active}} = \left| \bigcup_j [a_j, b_j] \right|, \qquad q_{\text{elapsed}} = \max_j b_j - \min_j a_j, \qquad q_{\text{work}} = \sum_j (b_j - a_j).$$

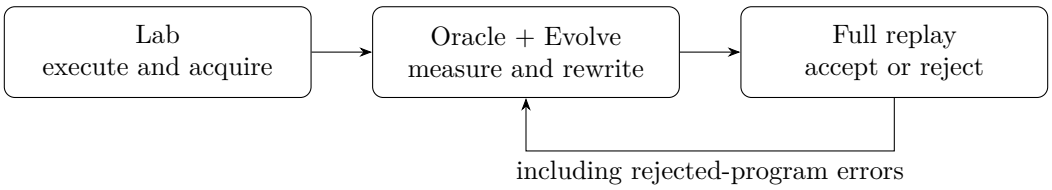
Overlap and idle gaps make these quantities differ even for identical records. Repeated delivery of one operation is not extra work. Time units must agree; request populations include failures as well as successes. To separate workload mix from within-class change, both periods use the before-period class weights:

$$\rho = \frac{\sum_g w_g^{\text{before}} \, \overline{q}_g^{\text{after}}}{\sum_g w_g^{\text{before}} \, \overline{q}_g^{\text{before}}}.$$

Ratios above 1.2 indicate regression, below 0.8 improvement, and otherwise stability, with declared numerical tolerances. These meanings and labels are supplied task definitions; the model authors their executable realization.

A record can establish duplicate delivery and changed workload mix while missing requests prevent a latency conclusion. Conversely, a measurement fault and a real regression can coexist. Collapsing the result to one fault label would erase independently supported evidence. Missing coverage, unknown clock units and conflicting operation records therefore remain explicit obligations.

2. Repair the computation, then reuse it. Lab executes the incumbent. Oracle receives its measured outputs and the independent evaluator's comparison. Evolve writes a complete replacement procedure. The next review must also receive a rejected candidate's execution errors: repeatedly showing incumbent failures cannot explain why a proposed repair never ran.



A proposal becomes a learner's experimental incumbent only when its whole-development score improves and no previously correct case regresses. Source, dataset, evaluator, model route and budget are frozen before inference. Every candidate and attempted call remains recorded. The numerical executor receives no answer key or generator case identifier. A restricted Python subset runs in a separate process with logged table acquisition and resource limits. It permits model-authored computations beyond a finite catalogue of supplied recipes; it is not a general-purpose security sandbox.

The retained source, scope statement and evidence references form one reusable object. Scope language is a model claim, not a proof; executable checks and counterexamples test it. For a renamed source, a model returns a field mapping. A fixed host transformation changes names only, preserving values and the frozen numerical source. This tests schema binding without smuggling in a new solver. It does not demonstrate discovery of an unfamiliar domain.

3. A campaign with separable claims. Three repeated temperature-zero learners receive at most six proposal rounds. They use the same model and initial material, not independent models or curricula. There are 64 development cases, 256 held-out cases, and 64 renamed-schema cases. Held-out fault combinations exclude those used in development; half require a numerical conclusion and half require unresolved checks. The factors themselves are authored and known. These are dependent synthetic tests of composition, not 256 independent real-world problems.

The first proposal is scored after freezing alongside the final program. Findings and unresolved sets must match exactly; ratios allow 1% relative or 0.005 absolute error. Deleting the learned object restores a naive summed-duration incumbent. An independent handwritten reference uses the same observed tables; the latent-data evaluator computes interval coverage by a different algorithm. On 32 paired cases, direct and prose-memory controls receive all raw tables. The latter gets a translation of the first learner's final algorithm. Neither has a code executor, so differences measure the entire compilation/execution arrangement, not memory alone. Prose equivalence is intended but not certified; the primary learner is chosen in advance, never by held-out score.

Researcher changes to syntax, prompts, providers or reasoning settings create new versions. Inspected cases become development evidence for subsequent design; fresh seeds do not erase knowledge of the task family. Rejected and malformed attempts remain failures. Current inexpensive Chinese-developed models are selected from verified catalogue and endpoint metadata, with serialized calls, a fixed spending ceiling and no expensive fallback.

4. Measured evidence. Five supplied prototypes reproduced locally with 198 passing tests. Their strong shared result was bounded executable investigation with applicability and counterexamples. Their searches used researcher-authored grammars, and simpler code controls matched key results. We retain those contributions without attributing their primitives to a language model.

The broad-feedback campaign used DeepSeek V4.1 Flash for 38 calls costing $0.0900. The predeclared primary learner's accepted development scores rose from 12/64 to 52/64 and then 60/64. Its first model proposal, rejected for four development regressions, nevertheless provides a one-shot comparator:

Repeated learner	First proposal	Frozen final	Renamed schema
1 (primary)	94/256	232/256	61/64
2	8/256	177/256	43/64
3	94/256	177/256	43/64

For the primary learner, development corrected 144 held-out answers and regressed six relative to its first proposal. The final program answered all 128 unresolved cases and 104/128 numerical cases. All remaining errors counted duplicate deliveries as additional billed work. On 32 paired cases, the frozen program answered 28 correctly; direct and prose controls each answered two. The handwritten reference answered 256/256; deleting the learned program restored 6/256. Execution tools differ between the model-answer controls and the program, so this does not isolate a memory advantage.

A follow-up reduced the witness context and exposed numerical residuals. Its frozen learners scored 170/256, 6/256 and 177/256: compact feedback did not establish better repair. One learner repeatedly ignored an actionable regression already present in its feedback. The final prose-control call was credit-rejected, so that campaign's controls remain incomplete. Across six versions, including three earlier failed attempts: 108 calls attempted, $0.1874 reported cost and $0.0088 reserved for the unpriced rejected call. Frozen programs, evaluators, inputs and individual outputs replay offline without inference.

Interpretation and next boundary. The promising direction is to spend language-model inference on acquiring a reusable computation and semantic binding, then run that computation cheaply on further observations. The necessary evidence is useful frozen reuse, accurate scope checks and retained competence as later procedures are added. A correct first program demonstrates compilation; improvement after feedback requires a separate comparison. Added examples and additional attempts can explain gains without a uniquely effective repair mechanism.

Program execution for model reasoning already appears in PAL [1]; executable skill accumulation appears in Voyager [2]; and library learning is central to DreamCoder [3]. Zenith does not claim those ideas as new. Its present question is narrower: can measured failures produce scoped investigations that compose supported findings without suppressing unresolved evidence? Equal-tool, equal-discovery-budget comparisons on new semantic contracts and real operational data are the next decisive tests. The retained financial studies establish no forecasting or causal advantage.

References

[1] L. Gao et al. *PAL: Program-aided Language Models.* 2022. arXiv:2211.10435.

[2] G. Wang et al. *Voyager: An Open-Ended Embodied Agent with Large Language Models.* 2023. arXiv:2305.16291.

[3] K. Ellis et al. *DreamCoder: Growing generalizable, interpretable knowledge with wake-sleep Bayesian program learning.* 2020. arXiv:2006.08381.